



Problem List



Submit



0



71:19:41



Premium

[Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

162. Find Peak Element

Solved

Medium

Topics

Companies

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`**Output:** `2`**Evaluation:** 2 is a peak element and your

14.9K



564



152 Online

</> Code

Java Auto

```
6 while (left < right) {
7     int mid = left + (right - left) / 2;
8
9     if (nums[mid] < nums[mid + 1]) {
10
11         left = mid + 1;
12     } else {
13
14         right = mid;
15     }
16 }
17
18
```

Saved

Ln 14, Col 29

☒ Testcase | Test Result

Accepted

Runtime: 0 ms

☒ Case 1☒ Case 2

Input

```
nums =
[1,2,3,1]
```



← All Submissions

Accepted 76 / 76 testcases passed

kanakkkkk_0708 submitted at Aug 14, 2026 18:49



Unlock the Full LeetCode Ex...

Company problems, Ask Leet, and expe...

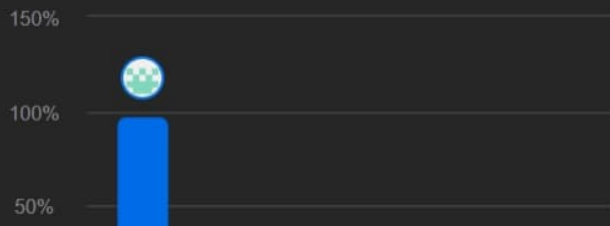


Runtime

0 ms | Beats 100.00% 🌟

Memory

43.90 MB | Beats 88.32% 🌟



</> Code

Submit Ctrl Enter

Java ▾ 🔒 Auto

```
6         while (left < right) {
7             int mid = left + (right - left) / 2;
8
9             if (nums[mid] < nums[mid + 1]) {
10
11                 left = mid + 1;
12             } else {
13
14                 right = mid;
15             }
16         }
17     }
18 }
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

nums =

[1, 2, 3, 1]



Problem List



Submit



0



71:19:41



Premium

[Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

33. Search in Rotated Sorted Array

Solved

Medium

Topics

Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of target* if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

30.6K



710



384 Online

</> Code

Java Auto

Push



```
1 class Solution {
2     public int search(int[] nums, int target) {
3         int low = 0, high = nums.length - 1;
4
5         while (low <= high) {
6             int mid = low + (high - low) / 2;
7
8             if (nums[mid] == target)
9                 return mid;
10
11             // Left half is sorted
12             if (nums[low] <= nums[mid]) {
```

Saved

Ln 1, Col 1

☒ Testcase | Test Result

Accepted

Runtime: 0 ms

☒ Case 1☒ Case 2☒ Case 3

Input

```
nums =
[4,5,6,7,0,1,2]
```



Problem List



Submit



Description | Accepted X | Editorial | S | <

< All Submissions



Accepted 197 / 197 testcases passed

kanakkkkk_0708 submitted at Aug 14, 2026 18:51



Unlock the Full LeetCode Ex...

Company problems, Ask Leet, and expe...



Runtime

0 ms | Beats 100.00% 🏆

Memory

43.61 MB | Beats 82.75% 🏆

150%

100%

50%



</> Code

Java ▾ 🔒 Auto

```
1 class Solution {
2     public int search(int[] nums, int target) {
3         int low = 0, high = nums.length - 1;
4
5         while (low <= high) {
6             int mid = low + (high - low) / 2;
7
8             if (nums[mid] == target)
9                 return mid;
10
11             // Left half is sorted
12             if (nums[low] <= nums[mid]) {
```

Saved

☑ Testcase | >_ Test Result

Accepted Runtime: 0 ms

☑ Case 1

☑ Case 2

☑ Case 3

Input

nums =

[4,5,6,7,0,1,2]

1464. Maximum Product of Two Elements in an Array

Easy

Topics

Companies

Hint

Given the array of integers `nums`, you will choose two different indices `i` and `j` of that array. Return the maximum value of $(\text{nums}[i]-1) * (\text{nums}[j]-1)$.

Example 1:

Input: `nums = [3,4,5,2]`

Output: 12

Explanation: If you choose the indices `i=1` and `j=2` (indexed from 0), you will get the maximum value, that is, $(\text{nums}[1]-1) * (\text{nums}[2]-1) = (4-1) * (5-1) = 3 * 4 = 12$.

Example 2:

Input: `nums = [1,5,4,5]`

Output: 16

2.8K



284



38 Online

</> Code

Java ▾ Auto

```
12         }
13     }
14
15     return (first - 1) * (second - 1);
16 }
17 }
18
19
```

Saved

☒ Testcase | > Test Result

Accepted

Runtime: 0 ms

☒ Case 1☒ Case 2☒ Case 3

Input

`nums =``[3,4,5,2]`

← All Submissions

Accepted 104 / 104 testcases passed

kanakkkkk_0708 submitted at Aug 14, 2026 18:53



₹583.25/mo

Save over 56%

Unlock the Full LeetCode Ex...

Company problems, Ask Leet, and expe...



Runtime

0 ms | Beats 100.00%

Memory

44.74 MB | Beats 33.23%

60%



40%

20%

Runtime: 5ms

Java

```
class Solution {  
    public int maxProduct(int[] nums) {  
        int result=0;  
        for(int i=0;i<nums.length;i++){  
            for(int j=i+1;j<nums.length;j++){  
                result=Math.max(result,(nums[i]-1) * (nums[j] -1));  
            }  
        }  
        return result;  
    }  
}
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =

[3,4,5,2]

1. Two Sum

Easy

Topics

Companies

Hint

You are given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to `target`*.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Java Auto

```
1 class Solution {
2     public int[] twoSum(int[] nums, int target) {
3         for (int i = 0; i < nums.length; i++) {
4             for (int j = i + 1; j < nums.length; j++) {
5                 if (nums[i] + nums[j] == target) {
6                     return new int[]{i, j};
7                 }
8             }
9         }
10    }
```

Saved

☒ Testcase | [Test Result](#)

Accepted

Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

`nums =`

`[2,7,11,15]`



Accepted 65 / 65 testcases passed

kanakkkkk_0708 submitted at Aug 14, 2026 18:57



Unlock the Full LeetCode Ex...

Company problems, Ask Leet, and exp...



Runtime

46 ms | Beats **14.01%**

Memory

46.74 MB | Beats **91.00%**

60%

40%

```
1 class Solution {
2     public int[] twoSum(int[] nums, int target) {
3         for (int i = 0; i < nums.length; i++) {
4             for (int j = i + 1; j < nums.length; j++) {
5                 if (nums[i] + nums[j] == target) {
6                     return new int[]{i, j};
7                 }
8             }
9         }
10    }
```

Saved

☒ Testcase | Test Result

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

nums =
[2,7,11,15]

26. Remove Duplicates from Sorted Array

Solved ✓

Easy

Topics

Companies

Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
```

19.6K



1.1K



452 Online

</> Code

Java Auto

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         if (nums.length == 0) return 0;
4
5         int i = 0;
6
7         for (int j = 1; j < nums.length; j++) {
8             if (nums[j] != nums[i]) {
9                 ...
10            }
11        }
12    }
13 }
```

Saved

☒ Testcase | >_ Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2

Input

```
nums =
[1,1,2]
```

Output

Description | Accepted × | Editorial | Solutions

← All Submissions

Accepted 362 / 362 testcases passed

 kanakkkkk_0708 submitted at Aug 14, 2026 18:58
₹583.25/mo
Save over 56% Unlock the Full LeetCode Ex...

Company problems, Ask Leet, and expe... →

⌚ Runtime

1 ms | Beats 77.41% 🌿

💾 Memory

46.26 MB | Beats 97.82% 🌿

75%

50%

25%

</> Code

Java ▾ 🔒 Auto

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         if (nums.length == 0) return 0;
4
5         int i = 0;
6
7         for (int j = 1; j < nums.length; j++) {
8             if (nums[j] != nums[i]) {
9                 ...
10            }
11        }
12    }
13 }
```

Saved

☑ Testcase | >_ Test Result

Accepted Runtime: 0 ms

☑ Case 1

☑ Case 2

Input

nums =
[1,1,2]

Output

628. Maximum Product of Three Numbers

Easy

Topics

Companies

Given an integer array `nums`, find three numbers whose product is maximum and return the maximum product.

Example 1:

Input: `nums = [1,2,3]`

Output: 6

Example 2:

Input: `nums = [1,2,3,4]`

Output: 24

Example 3:

Input: `nums = [-1,-2,-3]`

</> Code

Java Auto

```
31     }
32
33     return Math.max(max1 * max2 * max3, min1 * min2 * max1);
34 }
35 }
36
```

Saved

☒ Testcase | Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2☒ Case 3

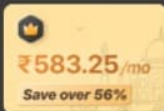
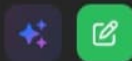
Input

`nums =`
`[1,2,3]`

← All Submissions

Accepted 65 / 65 testcases passed

kanakkkk_0708 submitted at Aug 14, 2026 18:57



Unlock the Full LeetCode Ex...

Company problems, Ask Leet, and expe...

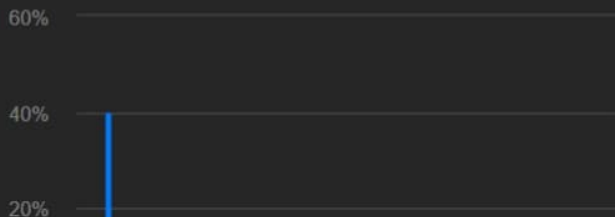


⌚ Runtime

46 ms | Beats 14.01%

💾 Memory

46.74 MB | Beats 91.00% 🏆



</> Code

Java ▾ 🔒 Auto

```
1 class Solution {
2     public int[] twoSum(int[] nums, int target) {
3         for (int i = 0; i < nums.length; i++) {
4             for (int j = i + 1; j < nums.length; j++) {
5                 if (nums[i] + nums[j] == target) {
6                     return new int[]{i, j};
7                 }
8             }
9         }
10    }
```

Saved

☑ Testcase | >_ Test Result

Accepted Runtime: 0 ms

☑ Case 1

☑ Case 2

☑ Case 3

Input

nums =

[2,7,11,15]

target =

```
1 class Solution {  
2     public int[] smallerNumbersThanCurrent(int[] nums) {  
3         int[] result = new int[nums.length];  
4  
5         for (int i = 0; i < nums.length; i++) {  
6             int count = 0;  
7  
8             for (int j = 0; j < nums.length; j++) {  
9                 if (nums[j] < nums[i]) {  
10                    count++;  
11                }  
12            }  
13  
14            result[i] = count;  
15        }  
    }  
}
```

Saved

☒ Testcase |  Test Result**Accepted**

Runtime: 0 ms

☒ Case 1☒ Case 2☒ Case 3

← All Submissions

Accepted 103 / 103 testcases passed

 kanakkkkk_0708 submitted at Aug 14, 2026 19:03



₹583.25/mo
Save over 56%

Unlock the Full LeetCode Ex...

Company problems, Ask Leet, and expe...



⌚ Runtime

8 ms | Beats 63.53% 🌱

💻 Memory

45.33 MB | Beats 83.53% 🌱

40%

20%

</> Code

Java ▾ 🔒 Auto

```

1  class Solution {
2      public int[] smallerNumbersThanCurrent(int[] nums) {
3          int[] result = new int[nums.length];
4
5          for (int i = 0; i < nums.length; i++) {
6              int count = 0;
7
8              for (int j = 0; j < nums.length; j++) {
9                  if (nums[j] < nums[i]) {
10                     count++;
11                 }
12             }
13
14             result[i] = count;
15         }
16     }
17 }
```

Saved

☒ Testcase | **>_ Test Result**

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Java   Auto

```
5
6     for (int i = 0; i < fruits.length; i++) {
7         boolean placed = false;
8
9         for (int j = 0; j < baskets.length; j++) {
10             if (baskets[j] >= fruits[i]) {
11                 // Use this basket
12                 baskets[j] = -1;
13                 placed = true;
14                 break;
15             }
16         }
17
18         if (!placed) {
19             unplaced++;
20         }
```

Saved

☒ Testcase |  Test Result**Accepted**

Runtime: 0 ms

☒ Case 1☒ Case 2

Description

Accepted × | Editorial | So

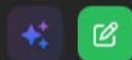
</> Code

Submit Ctrl Enter

← All Submissions

Accepted 904 / 904 testcases passed

kanakkkkk_0708 submitted at Aug 14, 2026 19:05



Unlock the Full LeetCode Ex...
Company problems, Ask Leet, and expe...



⌚ Runtime

1 ms | Beats 100.00% 🌟

💻 Memory

46.08 MB | Beats 96.88% 🌟

75%

50%

25%



Java ▾ 🔒 Auto

```
5
6     for (int i = 0; i < fruits.length; i++) {
7         boolean placed = false;
8
9         for (int j = 0; j < baskets.length; j++) {
10             if (baskets[j] >= fruits[i]) {
11                 // Use this basket
12                 baskets[j] = -1;
13                 placed = true;
14                 break;
15             }
16         }
17
18         if (!placed) {
19             unplaced++;
20         }
```

Saved

☒ Testcase | >_ Test Result

Accepted Runtime: 0 ms

☒ Case 1☒ Case 2

Input

121. Best Time to Buy and Sell Stock

Solved ✓

Easy

Topics

Companies

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return the *maximum profit* you can achieve from this transaction. If you cannot achieve any profit, return `0`.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: `5`

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6). profit = 6-1 =

</> Code

Java ▾ 🔒 Auto

```
1 class Solution {
2     public int maxProfit(int[] prices) {
3         int minPrice = Integer.MAX_VALUE;
4         int maxProfit = 0;
5
6         for (int i = 0; i < prices.length; i++) {
7
8             if (prices[i] < minPrice) {
9                 minPrice = prices[i];
10            }
11
12            else if (prices[i] - minPrice > maxProfit) {
13                maxProfit = prices[i] - minPrice;
14            }
15        }
```

Saved

☑ Testcase | >_ Test Result

Accepted Runtime: 0 ms

☑ Case 1

☑ Case 2

Input

Description
Accepted
Editorial

All Submissions

Accepted 212 / 212 testcases passed

kanakkkkk_0708 submitted at Aug 14, 2026 19:06

₹583.25/mo
Save over 56%

Unlock the Full LeetCode Ex...
Company problems, Ask Leet, and expe...

Runtime
1 ms | Beats 99.96%

Memory
94.59 MB | Beats 33.36%

Code
Submit Ctrl Enter

Java Auto

```

1 class Solution {
2     public int maxProfit(int[] prices) {
3         int minPrice = Integer.MAX_VALUE;
4         int maxProfit = 0;
5
6         for (int i = 0; i < prices.length; i++) {
7
8             if (prices[i] < minPrice) {
9                 minPrice = prices[i];
10            }
11
12            else if (prices[i] - minPrice > maxProfit) {
13                maxProfit = prices[i] - minPrice;
14            }
15        }
16    }
17 }

```

Saved

☒ Testcase Test Result

Accepted Runtime: 0 ms

☒ Case 1 Case 2

Input

C (gcc 5.4) ▾

🐱 Start Timer ▶



```
1 int getSecondLargest(int *arr, int n) {
2     int largest = INT_MIN;
3     int secondLargest = INT_MIN;
4
5     for (int i = 0; i < n; i++) {
6         int num = arr[i];
7
8         if (num > largest) {
9             secondLargest = largest;
10            largest = num;
11        } else if (num > secondLargest && num != largest) {
12            secondLargest = num;
13        }
14    }
15
16    |
17    return secondLargest;
18 }
19
```